```
#include <stdio.h>
#include <string.h>
#define MAX 100
struct Process {
    char pid[5];
    int arrival, burst, remaining;
    int completion, turnaround, waiting;
    int qno;
    int done;
};
int main() {
    struct Process p[MAX];
    int n, i, completed = 0;
    int current_time = 0, tq;
    float total_tat = 0, total_wt = 0;
    printf("1WA24CS218\n");
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf(p[i].pid, "p%d", i+1);
        printf("\nProcess %s\n", p[i].pid);
        printf("Enter Arrival Time: ");
        scanf("%d", &p[i].arrival);
        printf("Enter Burst Time: ");
        scanf("%d", &p[i].burst);
        p[i].remaining = p[i].burst;
        p[i].done = 0;
        printf("Enter Queue (1-System, 2-User): ");
        scanf("%d", &p[i].qno);
```

```
}
printf("\nEnter Time Quantum for System Queue: ");
scanf("%d", &tq);
while(completed < n) {
    int executed = 0;
    for(i = 0; i < n; i++) {
        if(p[i].qno == 1 && p[i].arrival <= current_time && p[i].remaining > 0) {
            executed = 1;
            if(p[i].remaining > tq) {
                p[i].remaining -= tq;
                current_time += tq;
            } else {
                current_time += p[i].remaining;
                p[i].remaining = 0;
                p[i].completion = current_time;
                p[i].turnaround = p[i].completion - p[i].arrival;
                p[i].waiting = p[i].turnaround - p[i].burst;
                total_tat += p[i].turnaround;
                total_wt += p[i].waiting;
                completed++;
            }
        }
    }
    if(executed)
        continue;
    int idx = -1;
    int min_arrival = 1e9;
    for(i = 0; i < n; i++) {
        if(p[i].qno == 2 && p[i].arrival <= current_time && p[i].remaining > 0) {
```

```
        if(p[i].arrival < min_arrival) {
            min_arrival = p[i].arrival;
            idx = i;
        }
    }
}
if(idx != -1) {
    executed = 1;
    int run_time = p[idx].remaining;
    for(int t = 1; t <= run_time; t++) {
        current_time++;
        p[idx].remaining--;
        int sys_arrived = 0;
        for(i = 0; i < n; i++) {
            if(p[i].qno == 1 && p[i].arrival <= current_time && p[i].remaining > 0) {
                sys_arrived = 1;
                break;
            }
        }
        if(sys_arrived)
            break;
    }
    if(p[idx].remaining == 0) {
        p[idx].completion = current_time;
        p[idx].turnaround = p[idx].completion - p[idx].arrival;
        p[idx].waiting = p[idx].turnaround - p[idx].burst;
        total_tat += p[idx].turnaround;
        total_wt += p[idx].waiting;
        completed++;
```

```
            }
        }
        if(!executed)
            current_time++;
    }
    printf("\nPID\tQ\tAT\tBT\tCT\tTAT\tWT\n");
    for(i = 0; i < n; i++) {
        printf("%s\t%d\t%d\t%d\t%d\t%d\t%d\n",
            p[i].pid, p[i].qno, p[i].arrival, p[i].burst,
            p[i].completion, p[i].turnaround, p[i].waiting);
    }
    printf("\nAverage Turnaround Time = %.2f", total_tat / n);
    printf("\nAverage Waiting Time = %.2f\n", total_wt / n);
    return 0;
}
```

```
1WA24CS218
Enter number of processes: 4

Process p1
Enter Arrival Time: 0
Enter Burst Time: 4
Enter Queue (1-System, 2-User): 1

Process p2
Enter Arrival Time: 0
Enter Burst Time: 3
Enter Queue (1-System, 2-User): 1

Process p3
Enter Arrival Time: 0
Enter Burst Time: 8
Enter Queue (1-System, 2-User): 2

Process p4
Enter Arrival Time: 10
Enter Burst Time: 5
Enter Queue (1-System, 2-User): 1

Enter Time Quantum for System Queue: 2

PID     Q       AT      BT      CT      TAT     WT
p1      1       0       4       6       6       2
p2      1       0       3       7       7       4
p3      2       0       8       20      20      12
p4      1       10      5       15      5       0

Average Turnaround Time = 9.50
Average Waiting Time = 4.50
```